

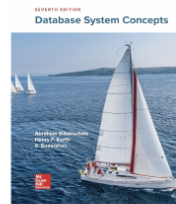
08.2: Map Reduce



UMD DATA605 - Big Data Systems

08.2: Map Reduce

- **Instructor:** Dr. GP Saggese, gsaggese@umd.edu
- @References@
 - Silberschatz: Chap 10
 - Ghemawat et al.: *The Google File System*, 2003
 - Dean et al.: *MapReduce: Simplified Data Processing on Large Clusters*, 2004



1 / 16

2 / 16: MapReduce: Overview

MapReduce: Overview

– **MapReduce programming model**

- Inspired by functional programming (e.g., Lisp)
- Common pattern of parallel programming to process large number of records

– **Basic algorithm**

- Apply `map()` to each record
- Group results by key
- Apply `reduce()` to results of `map()`

– **Example**

- *Goal*: Sum length of all tuples in a document
 - E.g.,
`[(), (a,) (a, b) (a, b, c)]`
- *map(function, set of values)*
 - Apply function to each value (e.g., `len`)
`map(len, [(), (a), (a, b), (a, b, c)]) -> [0, 1, 2, 3]`
- *reduce(function, set of values)*
 - Combine values using a binary function (e.g., `add`)
`reduce(add, [0, 1, 2, 3]) -> 6`

2 / 16

– **MapReduce programming model**

- The MapReduce model is inspired by *functional programming*, which is a style of programming where functions are treated as first-class citizens. Languages like Lisp have influenced this model.
- It is a common pattern used in *parallel programming*, which means it helps in processing a large number of records simultaneously. This is particularly useful when dealing with big data, as it allows for efficient data processing by distributing tasks across multiple machines.

– **Basic algorithm**

- The process begins with the `map()` function, which is applied to each record in the dataset. This function transforms the data into key-value pairs.
- After mapping, the results are grouped by key. This step is crucial as it organizes the data for the next phase.
- The `reduce()` function is then applied to the grouped data. This function aggregates the data, combining values associated with the same key to produce a final result.

– **Example**

- The goal of the example is to calculate the sum of the lengths of all tuples in a document. This demonstrates how MapReduce can be used for simple data aggregation tasks.
- The `map()` function is applied to each tuple to calculate its length. For instance, the tuple `(a, b, c)` has a length of 3.
- The `reduce()` function then takes these lengths and sums them up. In this example, the lengths `[0, 1, 2, 3]` are summed to produce a total of 6. This illustrates how MapReduce can efficiently process and summarize data.

3 / 16: MapReduce: Overview

MapReduce: Overview

- **Structure of computation**
 - *Read input*
 - Sequentially or in parallel
 - *Map*
 - Extract / compute from records
 - *Group by key*
 - Sort and shuffle
 - *Reduce*
 - Aggregate, summarize, filter, transform
 - *Write result*
- **Division of responsibilities**
 - User specifies `map()` and `reduce()` functions to solve problem
 - MapReduce framework (e.g., Hadoop, Spark) implements algorithm

3 / 16

- **MapReduce: Overview**
- **Structure of computation**
 - *Read input*: This is the first step where data is ingested into the system. It can be done either sequentially, where data is read one piece at a time, or in parallel, where multiple pieces of data are read simultaneously. This flexibility allows MapReduce to handle large datasets efficiently.
 - *Map*: In this phase, the data is processed to extract useful information or compute necessary values from each record. This is where the user-defined `map()` function comes into play, transforming the input data into a set of intermediate key-value pairs.
 - *Group by key*: After mapping, the framework sorts and shuffles the data so that all values associated with the same key are grouped together. This step is crucial for organizing the data before the reduction phase.
 - *Reduce*: Here, the grouped data is processed to aggregate, summarize, filter, or transform it into the final output. The user-defined `reduce()` function is applied to each group of values sharing the same key.
 - *Write result*: The final step involves writing the processed data to a storage system, making it available for further analysis or use.
- **Division of responsibilities**
 - Users are responsible for defining the `map()` and `reduce()` functions, which are tailored to solve specific problems by processing and transforming the data.
 - The MapReduce framework, such as Hadoop or Spark, handles the execution of the algorithm. It manages the distribution of data, parallel processing, and fault tolerance,

allowing users to focus on the logic of their data processing tasks without worrying about the underlying complexities.

4 / 16: MapReduce: Word Count

MapReduce: Word Count

- **Word Count**
 - “Hello world” of MapReduce
 - Huge text file (can't fit in memory)
 - Count occurrences of each distinct word
- **Linux solution**

```
> more doc.txt
One a penny, two a penny, hot cross buns.
> words doc.txt | sort | uniq -c
a 2
buns 1
cross 1
...
```

 - `words` outputs words one per line
 - Unix pipeline is parallelizable in MapReduce sense
- **Sample application**
 - Analyze web server logs for popular URLs

```
Hot cross buns!
Hot cross buns!
One a penny, two a penny,
Hot cross buns!

If you have no daughters,
Give them to your sons.
One a penny, two a penny,
Hot cross buns![1]
```

4 / 16

- **Word Count**
 - The “Hello world” of MapReduce refers to a simple introductory example used to demonstrate the basic functionality of the MapReduce programming model. In this case, it's about counting words.
 - The challenge is to handle a huge text file that cannot fit into memory, which is a common scenario in big data processing.
 - The goal is to count how many times each distinct word appears in the text.
- **Linux solution**
 - The example shows a simple Linux command-line solution using a Unix pipeline to achieve the word count.
 - The `words` command outputs each word on a new line, making it easier to process.
 - The pipeline uses `sort` to organize the words and `uniq -c` to count occurrences, demonstrating a parallelizable approach similar to MapReduce.
- **Sample application**
 - A practical application of this concept is analyzing web server logs to determine which URLs are most popular. This involves counting occurrences of each URL, similar to counting words in a text file.

5 / 16: MapReduce: Word Count

MapReduce: Word Count		
Action	Python code	Example
Read input	<code>values = read(file_name)</code>	"One a penny, two a penny, hot cross buns."
Map:	<pre>def map(values): # values: words in document for word in values: emit(word, 1)</pre>	Map:
- Invoke <code>map()</code> on each input record - Emit 0 or more output data items		<pre>[("one", 1), ("a", 1), ("penny", 1), ("two", 1), ("a", 1), ("penny", 1), ("hot", 1), ("cross", 1), ("buns", 1)]</pre>
Group by key:	<pre>def reduce(key, values): # key: a word # value: a list of counts result = 0 # result = sum(values) for count in values: result += count emit(key, result)</pre>	Group by key:
- Gather all outputs from <code>map()</code> stage - Collect outputs by keys		<pre>[("a", [1, 1]), ("buns", [1]), ("cross", [1]), ("hot", [1]), ("one", [1]), ("penny", [1, 1]), ("two", [1])]</pre>
Reduce:		Reduce:
Combine the list of outputs with same keys		<pre>[("one", 1), ("a", 2), ("penny", 2), ("two", 1), ("hot", 1), ("cross", 1), ("buns", 1)]</pre>

5 / 16

– MapReduce: Word Count

This slide explains how the MapReduce programming model can be used to perform a word count operation. MapReduce is a framework for processing large data sets with a distributed algorithm on a cluster.

– Action

- *Read input:* The process begins by reading the input data, which is typically a large text file or dataset.
- **Map:** The `map()` function is applied to each input record. It processes the data and emits key-value pairs. In this case, each word in the document is emitted with a count of 1.
- **Group by key:** After the map phase, all emitted key-value pairs are gathered and grouped by key. This means all occurrences of the same word are collected together.
- **Reduce:** The `reduce()` function takes each key and its associated list of values (counts) and combines them. For word count, this means summing up the counts for each word.

– Python Code

- The code snippet shows how to implement the MapReduce word count in Python.
- `read(file_name)`: Reads the input file and returns the data.
- `map(values)`: Iterates over each word in the input, emitting a key-value pair of the word and the number 1.
- `reduce(key, values)`: Takes a word (key) and a list of counts (values), sums the counts, and emits the total count for each word.

– Example

- The example uses the phrase “One a penny, two a penny, hot cross buns.”
- **Map:** Each word is paired with the number 1, resulting in a list of key-value pairs.
- **Group by key:** Words are grouped together, showing how many times each word appears.
- **Reduce:** The counts for each word are summed, resulting in the final word count. For instance, the word “a” appears twice, so its count is 2.

6 / 16: MapReduce: Log Processing

MapReduce: Log Processing

- **Goal:**
 - Log file records website access
(date, hour, filename)
 - Count file accesses in Feb 2013
- **Input**
 - Read file, split into lines
- **Map**
 - Parse line into 3 fields
 - If date is in interval,
emit(dir_name, 1)
- **GroupBy**
 - Reduce key: filename
 - Accumulate (key, value) with
same filename
- **Reduce**
 - Sum values for each (key, value)
list with same filename
 - Output file access count
- **Output**
 - Write results to disk.

```

After Input
2013/02/21 10:31:22.00EST /slide-dir/1
2013/02/21 10:43:12.00EST /slide-dir/1
2013/02/22 18:26:45.00EST /slide-dir/1
2013/02/22 18:26:48.00EST /exer-dir/2
2013/02/22 18:26:54.00EST /exer-dir/3
2013/02/22 20:53:29.00EST /slide-dir/1

After Map
['/slide-dir/11.ppt', 1), ...]

After GroupBy
[['/slide_dir/11.ppt', 1), ...,
 ('/slide-dir/12.ppt', [1, 1]), ...]

After Reduce
[['/slide_dir/11.ppt', 1), ...,
 ('/slide-dir/12.ppt', 2), ...]

Output
/slide_dir/11.ppt 1
...
/slide-dir/12.ppt 2
...

```

6 / 16

- **Goal:**
 - The main objective here is to process log files that record website access. Each log entry contains information about the date, hour, and filename accessed.
 - Specifically, we want to count how many times each file was accessed during February 2013. This is a common task in web analytics to understand user behavior and resource demand.
- **Input:**
 - The process begins by reading the log file and splitting it into individual lines. Each line represents a single access event.
- **Map:**
 - In the mapping phase, each line is parsed into three fields: date, hour, and filename.
 - If the date falls within February 2013, the map function emits a key-value pair where the key is the directory name and the value is 1. This indicates a single access event for that file.
- **GroupBy:**
 - The grouping phase organizes the data by filename. All entries with the same filename are grouped together.

- This step is crucial as it prepares the data for the reduction phase by collecting all access counts for each file.
- **Reduce:**
 - During the reduction phase, the values for each grouped key (filename) are summed up.
 - This results in the total count of accesses for each file, which is the desired output.
- **Output:**
 - Finally, the results are written to disk. Each line in the output file contains a filename followed by the number of times it was accessed, separated by a newline.

The right column provides a step-by-step illustration of how the data transforms through each phase. After the input phase, you see raw log entries. The map phase shows how each entry is converted into a key-value pair. GroupBy organizes these pairs by filename, and Reduce sums the values to produce the final count. The output is a simple list of filenames with their respective access counts.

7 / 16: MapReduce: Interfaces

MapReduce: Interfaces

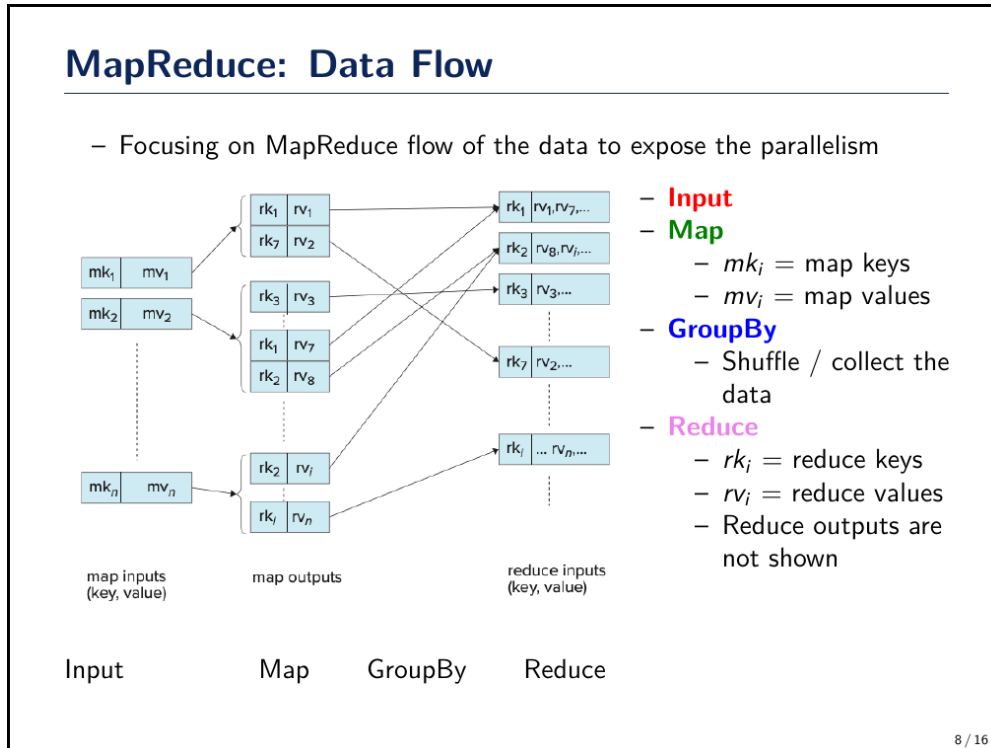
- **Input:** Read key-value pairs `List[Tuple[k, v]]`
- **Programmer** specifies two methods `map` and `reduce`
- **Map**
 - `Map(Tuple[k, v]) → List[Tuple[k, v]]`
 - Take a key-value pair and output a set of key-value pairs
 - E.g., key is a file, value is occurrences
 - “One a penny” → `[("One", 1), ("a", 1), ("penny", 1)]`
 - One `Map` call for every `(k, v)` pair
- **GroupBy**
 - `GroupBy(List[Tuple[k, v]]) → List[Tuple[k, List[v]]]`
 - Group and optionally sort records with the reduce key
- **Reduce**
 - `Reduce(Tuple[k, List[v]]) → Tuple[k, v]`
 - Reduce all values `v'` with same key `k'`
 - One `Reduce` call per unique key `*k'`

7 / 16

- **MapReduce: Interfaces**
 - **Input:** The process begins by reading data in the form of key-value pairs. This is essentially a list where each item is a tuple consisting of a key and a value. For example, if processing text files, the key might be the file name, and the value could be the content or occurrences of words within that file.
 - **Programmer:** The programmer’s role is to define two essential methods: `map` and `reduce`. These methods dictate how data is processed and transformed throughout the MapReduce operation.
 - **Map:**
 - The `map` function takes a single key-value pair and transforms it into a list of new

- key-value pairs. For instance, if the input is a line of text, the map function might break it down into individual words, each paired with the number 1, indicating a single occurrence.
- Each key-value pair from the input triggers one call to the **map** function, ensuring that every piece of data is processed.
- **GroupBy:**
 - After mapping, the **GroupBy** function organizes the data by keys. It collects all values associated with the same key into a list. This step is crucial for the subsequent reduction process, as it prepares the data for aggregation.
- **Reduce:**
 - The **reduce** function takes each unique key and its associated list of values, then processes them to produce a single output key-value pair. This step typically involves aggregating or summarizing the data, such as summing occurrences of words.
 - Each unique key results in one call to the **reduce** function, ensuring that all related data is combined appropriately.
- **Output:** The final step is to write the processed data back out as key-value pairs. This output can then be used for further analysis or storage, completing the MapReduce cycle.

8 / 16: MapReduce: Data Flow



- **MapReduce Data Flow**
 - This slide illustrates the data flow in the MapReduce framework, emphasizing how parallelism is achieved.
- **Input**
 - The process begins with input data, which is structured as key-value pairs. Each pair is

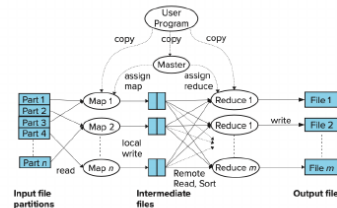
denoted as (mk_i) (map key) and (mv_i) (map value).

- **Map**
 - The map function processes each input pair independently, producing intermediate key-value pairs. These are also denoted as (mk_i) and (mv_i).
- **GroupBy**
 - This stage involves shuffling and sorting the intermediate data. The goal is to group all values associated with the same key together, preparing them for the reduce phase.
- **Reduce**
 - In the reduce phase, the grouped data is processed to produce the final output. Each group is identified by a reduce key (rk_i) and contains multiple reduce values (rv_i).
 - The reduce function aggregates or processes these values to produce the final output, although the outputs are not shown in this diagram.
- **Parallelism**
 - The diagram highlights how each stage can be executed in parallel, allowing for efficient processing of large datasets. Each map and reduce task can run independently, leveraging distributed computing resources.

9 / 16: MapReduce: Parallel Data Flow

MapReduce: Parallel Data Flow

- **User program** specifies map/reduce code
 - *MasterNode* sends code to all computing nodes
 - *Machines* reused for multiple computations (**Map**, **Reduce**) at different times
 - All operations use HDFS as storage
- **Map**
 - n data chunks to process
 - Functions executed in parallel on k machines
 - Output data saved on disk
- **GroupBy / Sort**
 - Output data sorted and partitioned by reduce key
 - Files created for each **Reduce** task
- **Reduce**
 - Functions executed in parallel on multiple machines
 - Each works on part of the data
 - Output data saved on disk



9 / 16

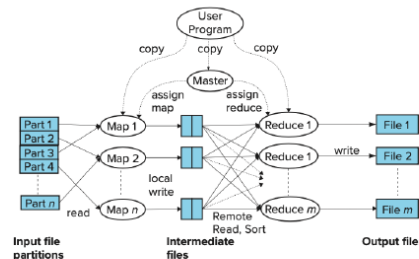
- **User Program:**
 - The user specifies the code for the **map** and **reduce** functions. This code is crucial as it defines how data will be processed and transformed.
 - The *MasterNode* is responsible for distributing this code to all computing nodes, ensuring that each node knows what task to perform.
 - The same machines are used for different computations, such as **Map** and **Reduce**, at various times, optimizing resource usage.

- All operations rely on the Hadoop Distributed File System (HDFS) for storage, which provides reliable and distributed data storage.
- **Map:**
 - The input data is divided into n chunks, each processed independently.
 - These chunks are processed in parallel across k machines, enhancing efficiency and speed.
 - The output from each **Map** function is saved to disk, ready for the next stage.
- **GroupBy / Sort:**
 - The output data from the **Map** phase is sorted and partitioned based on the reduce key.
 - This step organizes the data into files, each corresponding to a specific **Reduce** task, ensuring that related data is processed together.
- **Reduce:**
 - The **Reduce** functions are executed in parallel on multiple machines, similar to the **Map** phase.
 - Each machine handles a portion of the data, allowing for distributed processing.
 - The final output from the **Reduce** phase is saved to disk, resulting in the creation of output files that contain the processed data.

10 / 16: MasterNode Responsibilities

MasterNode Responsibilities

- **MasterNode coordinates / schedule tasks**
 - Task status: idle, in-progress, completed
 - Schedule idle tasks as workers become available
 - **Map** task completion sends location and sizes of intermediate files to Master
 - Master informs **Reduce** tasks
 - Schedule idle **Reduce** tasks
- **MasterNode pings workers to detect failures**
 - Heartbeat



10 / 16

- **MasterNode Responsibilities**
 - The *MasterNode* is crucial in managing and coordinating tasks within a distributed computing environment.
 - It **coordinates and schedules tasks** by keeping track of their status: idle, in-progress, or completed.
 - When tasks are idle, the *MasterNode* schedules them as workers become available, ensuring efficient resource utilization.

- Upon completion of a **Map** task, it sends the location and sizes of intermediate files back to the MasterNode.
 - This information is essential for the MasterNode to inform and schedule **Reduce** tasks.
- The MasterNode is responsible for scheduling idle **Reduce** tasks, ensuring they start processing as soon as possible.
- **Failure Detection**
 - The MasterNode uses a **heartbeat mechanism** to ping workers and detect failures.
 - This helps maintain system reliability by identifying and addressing worker failures promptly.

The diagram illustrates the flow of data and tasks between the MasterNode, **Map** tasks, and **Reduce** tasks, highlighting the MasterNode's role in task assignment and coordination.

11 / 16: Dealing with Failures

Dealing with Failures

- **Map worker failure**
 - Reset failed map tasks to idle
 - Notify reduce workers when task is rescheduled
- **Reduce worker failure**
 - Reset in-progress tasks to idle
 - Restart reduce task
- **Master failure**
 - Abort MapReduce task
 - Notify client

11 / 16

- **Dealing with Failures**
- **Map worker failure**
 - When a map worker fails, any tasks it was handling are reset to an *idle* state. This means they are marked as not started, so they can be picked up by other workers. This is important because it ensures that no data is lost and the process can continue smoothly.
 - Reduce workers, which depend on the output of map tasks, need to be informed when a map task is rescheduled. This notification helps them know that they should expect new data from the rescheduled task, ensuring they work with the most up-to-date information.

- **Reduce worker failure**

- If a reduce worker fails, any tasks it was working on are also reset to an *idle* state. This allows other workers to take over these tasks, ensuring that the process can continue without interruption.
- The reduce task is then restarted, which means it begins again from the start. This is crucial to ensure that all data is processed correctly and no results are missed.

- **Master failure**

- The master node is responsible for coordinating the entire MapReduce process. If it fails, the whole task is aborted because the master node is essential for managing task distribution and progress tracking.
- The client, who initiated the MapReduce task, is notified of the failure. This notification is important so the client knows that the task did not complete and can take appropriate action, such as restarting the task or investigating the cause of the failure.

12 / 16: How Many Map and Reduce Jobs?

How Many Map and Reduce Jobs?

- Number of:
 - Map tasks: M
 - Reduce tasks: R
 - Worker nodes: N
- Typically $M \gg N$
 - Pros:
 - Improve dynamic load balancing
 - Speed up recovery from worker failures
 - Cons:
 - More communication between *MasterNode* and *WorkerNodes*
 - Lots of smaller files
- Typically $R > N$
- Usually $R < M$, output is spread across fewer files

12 / 16

- **Number of Map tasks (M):** This refers to the number of tasks that are responsible for processing chunks of the input data. Each task processes a portion of the data independently.
- **Number of Reduce tasks (R):** These tasks take the output from the map tasks and perform a summary operation, such as counting or averaging.
- **Worker nodes (N):** These are the machines in a cluster that execute the map and reduce tasks. Each node can handle multiple tasks, but the total number of tasks often exceeds the number of nodes.

- Typically $M \gg N$:
 - **Pros:**
 - *Improve dynamic load balancing*: By having more map tasks than worker nodes, the system can distribute tasks more evenly across nodes, which helps in balancing the load dynamically.
 - *Speed up recovery from worker failures*: If a worker node fails, having many small tasks allows the system to reassign the tasks to other nodes quickly, minimizing downtime.
 - **Cons:**
 - *More communication between MasterNode and WorkerNodes*: With more tasks, the master node has to manage and communicate with worker nodes more frequently, which can increase overhead.
 - *Lots of smaller files*: Each map task might produce its own output file, leading to a large number of small files, which can be inefficient to manage.
- Typically $R > N$: This setup ensures that the reduce tasks can be distributed across the available nodes, allowing for parallel processing and efficient use of resources.
- Usually $R < M$: This means that the output from the map tasks is consolidated into fewer files by the reduce tasks, which helps in managing the output more efficiently and reduces the number of files to handle.

13 / 16: Refinements: Backup Tasks

Refinements: Backup Tasks

- **Problem**
 - Slow workers significantly lengthen the job completion time
 - Slow workers due to:
 - Older processor
 - Not enough RAM
 - Other jobs on the machine
 - Bad disks
 - OS thrashing / virtual memory hell
- **Solution**
 - Near the end of Map / Reduce phase
 - Spawn backup copies of tasks
 - Whichever one finishes first “wins”
- **Result**
 - Shorten job completion time

13 / 16

- **Refinements: Backup Tasks**
- **@Problem@**

- The main issue here is that *slow workers* can cause a significant delay in completing a job. This is a common problem in distributed computing where tasks are spread across multiple machines.
- Slow workers can be caused by several factors:
 - **Older processor:** Machines with outdated processors may not handle tasks as efficiently as newer ones.
 - **Not enough RAM:** Insufficient memory can slow down processing as the machine struggles to handle data.
 - **Other jobs on the machine:** If a machine is running multiple tasks, it can become overloaded, slowing down all processes.
 - **Bad disks:** Faulty or slow storage can bottleneck data access and processing.
 - **OS thrashing / virtual memory hell:** When a system uses too much virtual memory, it can lead to excessive paging, slowing down the entire system.
- ****@Solution@****
 - To address this, a strategy is employed near the end of the **Map** or **Reduce** phase of a job.
 - The idea is to *spawn backup copies* of tasks that are running slowly. This means creating duplicate tasks on different machines.
 - The first task to complete successfully is the one that “wins,” meaning its results are used, and the others are discarded.
- ****@Result@****
 - By implementing this strategy, the overall job completion time is reduced. This is because the system is no longer held up by the slowest workers, as backup tasks ensure that at least one task finishes quickly.

14 / 16: Refinement: Combiners

Refinement: Combiners

- **Problem**
 - Often a **Map** task produces many pairs for the same key **k**
[(k1, v1), (k1, v2), ...]
 - E.g., common words in the word count example
 - Increase complexity of the **GroupBy** stage
- **Solution**
 - Pre-aggregate values in the **Map** with a **Combine**
[k1, (v1, v2, ...), k2, ([...])]
 - **Combine** is usually the same as the **Reduce** function
 - Works only if **Reduce** function is commutative and associative
- **Result**
 - Better data locality
 - Less shuffling and reordering
 - Less network / disk traffic

14 / 16

- ****@Problem@****
 - When a **Map** task runs, it often generates many key-value pairs where the key is the same. For example, in a word count task, the word “the” might appear many times, leading to pairs like [(k1, v1), (k1, v2), ...]. This can make the **GroupBy** stage more complex because it has to handle a large number of pairs for the same key.
 - This is particularly noticeable with common words in text processing tasks, where certain keys (words) appear very frequently.
- ****@Solution@****
 - To address this, we can use a **Combine** function during the **Map** phase to pre-aggregate values. This means that instead of sending all individual pairs to the next stage, we combine them into a single pair with a list of values, like [k1, (v1, v2, ...), k2, ([...])].
 - The **Combine** function is typically the same as the **Reduce** function, which processes the data in the final stage. However, this approach only works if the **Reduce** function is *commutative* (order doesn’t matter) and *associative* (grouping doesn’t matter).
- ****@Result@****
 - Using combiners improves data locality, meaning that data is processed closer to where it is stored, reducing the need to move data around.
 - It reduces the amount of shuffling and reordering of data between the **Map** and **Reduce** stages, which can be a costly operation.
 - This leads to less network and disk traffic, making the overall process more efficient and faster.

15 / 16: Refinement: Partition Function

Refinement: Partition Function

- **Problem**
 - Users want to control key partitioning
 - Inputs to **Map** tasks created by contiguous input file splits
 - Default partition function: `hash(key) mod R`
 - Ensure records with the same intermediate key go to the same worker
- **Solution**
 - Override hash function:
 - E.g., `hash(hostname(URL)) mod R` ensures URLs from a host end up in the same output file

15 / 16

- Refinement: Partition Function

- ****@Problem@****

- *Users want to control key partitioning:* In distributed computing, especially in systems like Hadoop, data is split across multiple nodes. Users often need control over how data is partitioned to optimize processing.
- *Inputs to **Map** tasks created by contiguous input file splits:* When processing large datasets, files are split into chunks, and each chunk is processed by a separate **Map** task. This ensures parallel processing but can lead to scattered data if not managed properly.
- *Default partition function: `hash(key) mod R`:* By default, data is partitioned using a hash function. This function takes a key, hashes it, and then takes the modulus with the number of reducers (**R**). This method distributes data evenly but may not group related data together.
- *Ensure records with the same intermediate key go to the same worker:* It's crucial that all data with the same key ends up on the same node to ensure correct and efficient processing.

- ****@Solution@****

- *Override hash function:* Users can customize the partitioning by overriding the default hash function. This allows for more control over how data is grouped and processed.
- *E.g., `hash(hostname(URL)) mod R` ensures URLs from a host end up in the same output file:* By customizing the hash function to consider specific attributes, like the hostname of a URL, users can ensure that related data (e.g., all URLs from the same host) is processed together, improving efficiency and relevance of the output.

16 / 16: Implementations of MapReduce

Implementations of MapReduce

- There are many implementations of MapReduce:
 - **Google**
 - Proprietary implementation
 - **Hadoop**
 - Open-source in Java
 - Uses HDFS for storage
 - Used by Yahoo, Facebook for large-scale data processing
 - **Amazon Elastic MapReduce (EMR)**
 - Hadoop MapReduce on Amazon EC2
 - Supports Spark, HBase, Hive, other big data frameworks
 - Used for processing large datasets on AWS
 - **Spark**
 - Open-source cluster-computing framework
 - Known for speed, ease of use
 - Used by Netflix for real-time analytics
 - **Dask**
 - Parallel computing with task scheduling
 - Integrates with Python's ecosystem
 - Suitable for large datasets on a single machine or cluster

16 / 16

- Implementations of MapReduce

- ****@Google@****
 - Google was the original creator of the MapReduce programming model. Their implementation is proprietary, meaning it is not available for public use. It was designed to handle large-scale data processing across distributed systems, which was a revolutionary approach at the time.
- ****@Hadoop@****
 - Hadoop is an open-source implementation of MapReduce, written in Java. It is widely used because it allows for the processing of large data sets across clusters of computers using simple programming models. Hadoop uses the Hadoop Distributed File System (HDFS) for storage, which is designed to scale up from single servers to thousands of machines. Companies like Yahoo and Facebook have used Hadoop for their large-scale data processing needs.
- ****@Amazon Elastic MapReduce (EMR)@****
 - Amazon EMR is a cloud-based service that provides a managed Hadoop framework to process vast amounts of data across dynamically scalable Amazon EC2 instances. It supports various big data frameworks like Spark, HBase, and Hive, making it versatile for different data processing tasks. EMR is particularly useful for organizations that want to leverage AWS infrastructure for big data processing without managing the underlying hardware.
- ****@Spark@****
 - Apache Spark is an open-source cluster-computing framework known for its speed and ease of use. Unlike traditional MapReduce, Spark can perform in-memory data processing, which significantly speeds up data analytics tasks. It is used by

companies like Netflix for real-time analytics, allowing them to process and analyze data quickly and efficiently.

- ****@Dask@****

- Dask is a parallel computing library in Python that provides advanced parallelism for analytics, enabling computations on large datasets. It integrates well with Python's ecosystem, making it a good choice for Python developers. Dask can be used on a single machine or a cluster, making it flexible for different scales of data processing tasks.